

WINMOL segmentor — open pull requests

Four PRs open against main · status 2026-08-05

At a glance

PR	What it does	Size	State
#15	CPU speedup (~10x) + parity/latency docs	26 files	READY
#16	Synthetic data generation + RGBD input	66 files	READY
#13	BAMFORESTS benchmark + multiscale loader	17 files	draft
#14	Learned stem vectorization study	37 files	draft

Consolidated 2026-08-05: #12 merged into #15 · #17 merged into #16, then closed.

Suggested merge order

- #15 — 103 tests pass; merge NORMALLY (not squash) so #12 auto-closes as merged
- #16 — self-contained; the RGBD contract change is backwards compatible
- #13 / #14 — leave as drafts; both need their result artifacts committed

training/dataset.py is touched by both #13 (resize=) and #16 (depth_dir=).

They fork from the same main commit and neither is an ancestor of the other, so whichever merges second needs a manual reconcile there.

The parameters compose cleanly; git will not do it automatically.

#15 — CPU-inference speedup + parity docs

READY · +1688 across 26 files · full suite 103 passed · contains #12

What it does

Two independent levers for faster UNet inference that keep the frozen ONNX contract: a `width_mult` scaling every channel width (1.0 byte-identical to the historical ladder), and int8 quantization (static QDQ, calibrated on real tiles). Ships a Keras->ONNX converter for the Zenodo flavours and a release script. Reported ~10x CPU speedup at unchanged F1.

Two real bugs found in review — both would have shipped

- `width_mult` outside {1.0, 0.5, 0.25} crashed at forward time: decoder blocks
- took input channels from the next ladder width, not the actual skip+upsample
- concat. The CLI accepted arbitrary floats.
- Dynamic int8 export was unloadable: `quantize_dynamic(QInt8)` emits `ConvInteger`
- with signed weights, which onnxruntime's CPU EP does not implement — session
- load failed on the branch's own minimum onnxruntime. Fixed to `QUInt8`. The
- static path behind the 10x claim was unaffected.

#16 (part 1) — Depth simulator + RGBD input

READY · now also contains the synthetic-data work from #17

What it does

- `simulate_depth.py` synthesizes terrain depth from existing stem masks: fractal
- terrain + cylindrical stem bulges, deliberately degraded with bulge dropout,
- off-mask distractors and noise so depth is a helpful-but-unreliable cue.
- `--mismatch` pairs each tile with the WRONG mask as a leakage control.
- `--rgbd`: channels-parameterized ONNX contract (`IN_CHANNELS` stays 3),
- `normalize_depth`, `in_channels` through factory and export, a channel-inferring
- `OnnxSegmenter`, optional `depth_dir` in `StemDataset`, depth as geometric-only aug.

Result on held-out real TestDS

arm	F1	reading
RGB baseline	0.738	—
RGBD, matched depth	0.892	+0.15
RGBD, mismatched depth	0.755	~ RGB

The mismatch control is what makes this credible: the gain is genuine fusion, not the model reading the label off the fourth channel.

#16 (part 2) — Synthetic training data generation

was #17, merged into #16 and closed · 66 files total

Headline

F1 0.693 on real TestDS from purely generated imagery

94% of the 0.738 that real training data achieves — with zero manual annotation.

Pipeline

seeded SceneSpec -> aligned labels (binary mask, per-stem instances, amodal instances + visibility, 16-bit height field) -> SDXL ControlNet trained on 3,681 real SpecDS mask->tile pairs paints the photorealistic RGB.

What the evidence showed

- Label fidelity beats visual realism. The most realistic generator was the
- worst teacher (F1 0.195, recall 0.112) — it painted brash over labelled stems.
- Prompting cannot move appearance; data can. A ControlNet trained on one
- domain overrides the text prompt almost entirely.
- Instance segmentation needs a box-free method. Mask R-CNN reached 0.079; a
- stem fills a median 12.9% of its axis-aligned box, so the mask head paints a
- thin diagonal inside a box that is ~87% background.
- Real depth is supported robustly: nodata handling, fixed physical ranges, and
- a validate_dataset.py preflight reporting the depth->mask AUC.

#13, #14 — the research drafts

kept open for the record; neither is ready to merge

#13 — BAMFORESTS benchmark + multiscale loader (draft)

Native-resolution multiscale training, deterministic grid-tiled eval, a site-held-out benchmark harness, and ONNX execution-provider selection.

Dataloader and runtime code are well tested (7 + 5 tests). The benchmark claims are prose-only: cited results/*.md artifacts are not in the PR, single-seed.

To un-draft: smoke tests for the two scripts, plus the result artifacts.

#14 — Learned stem vectorization (draft)

Distills the analyzer's heuristic mask->graph vectorization into a multi-head UNet predicting centreline / orientation / diameter fields.

Newly relevant: #17's instance result is negative and points back here — box detection fails on long diagonal stems; this formulation avoids boxes entirely.

Merge mechanics worth knowing

#12's head commit is a true ancestor of #15 (merged, not cherry-picked), so a normal merge lets GitHub close #12 as MERGED automatically. A squash-merge rewrites that history and #12 would have to be closed by hand.

training/dataset.py is touched by both #13 (resize=) and #16 (depth_dir=).

They fork from the same main commit and neither is an ancestor of the other, so whichever merges second needs a manual reconcile. The parameters compose.